

Resumen de Variables, Tipos de Datos y Estructuras de Control en Python

1. Variables

- **Definición:** Espacios en memoria donde se almacenan valores. No es necesario declarar el tipo de la variable; Python infiere el tipo automáticamente.
- **Asignación:** Se asigna un valor a una variable utilizando el signo `=`. Ejemplo: `x = 10`.

2. Tipos de Datos

- **Primitivos:**
 - **Enteros (int):** Números enteros. Ejemplo: `x = 5`.
 - **Flotantes (float):** Números con decimales. Ejemplo: `y = 3.14`.
 - **Cadenas de texto (str):** Secuencias de caracteres. Ejemplo: `nombre = "Juan"`.
 - **Booleanos (bool):** Valores lógicos `True` o `False`. Ejemplo: `es_mayor = True`.
- **Colecciones:**
 - **Listas (list):** Secuencias ordenadas y mutables de elementos. Ejemplo: `mi_lista = [1, 2, 3]`.
 - **Tuplas (tuple):** Secuencias ordenadas e inmutables. Ejemplo: `mi_tupla = (1, 2, 3)`.
 - **Conjuntos (set):** Colección desordenada de elementos únicos. Ejemplo: `mi_conjunto = {1, 2, 3}`.
 - **Diccionarios (dict):** Colección de pares clave-valor. Ejemplo: `mi_diccionario = {"clave": "valor"}`.

3. Estructuras de Control

- **Condicionales:**
 - **if, elif, else:** Permiten tomar decisiones. Ejemplo:

```
if x > 0:
    print("Positivo")
elif x < 0:
    print("Negativo")
else:
    print("Cero")
```

- **Bucles:**
 - **for:** Itera sobre una secuencia (lista, tupla, cadena). Ejemplo:

```
for i in range(5):
    print(i)
```

- **while:** Repite mientras una condición sea verdadera. Ejemplo:

```
while x > 0:
    print(x)
    x -= 1
```

- **Control de bucles:**

- **break:** Termina el bucle de forma anticipada.
- **continue:** Salta a la siguiente iteración del bucle.
- **pass:** No hace nada, sirve como marcador de posición.

Resumen sobre Funciones en Python

1. Concepto

- **Definición:** Una función es un bloque de código reutilizable que realiza una tarea específica. Las funciones permiten modularizar y organizar el código, facilitando su mantenimiento y reutilización.
- **Sintaxis básica:**

```
def nombre_funcion(parámetros):
    # Cuerpo de la función
    return valor
```

2. Parámetros

- **Definición:** Los parámetros son las entradas que una función puede recibir para realizar su tarea. Se especifican entre paréntesis al definir la función.
- **Tipos de parámetros:**
 - **Posicionales:** Se pasan en un orden específico. Ejemplo:

```
def sumar(a, b):
    return a + b

resultado = sumar(3, 4)
```

- **Por defecto:** Tienen un valor predeterminado que se usa si no se proporciona un argumento. Ejemplo:

```
def saludar(nombre="Juan"):
    print(f"Hola, {nombre}")

saludar() # Imprime "Hola, Juan"
saludar("María") # Imprime "Hola, María"
```

- **Arbitrarios:** Se usan cuando no se conoce de antemano cuántos argumentos se pasarán. Se utilizan `*args` para parámetros posicionales arbitrarios y `**kwargs` para parámetros con nombre arbitrarios. Ejemplo:

```
def suma(*numeros):
    return sum(numeros)

resultado = suma(1, 2, 3, 4) # Devuelve 10
```

3. Invocación

- **Definición:** Llamar o ejecutar una función previamente definida, proporcionando los argumentos necesarios.
- **Ejemplo:**

```
def multiplicar(a, b):
    return a * b

resultado = multiplicar(5, 3) # Invoca la función y guarda el resultado
```

4. Recursividad

- **Definición:** Una función es recursiva cuando se llama a sí misma dentro de su propio código. Es útil para resolver problemas que pueden dividirse en subproblemas más pequeños del mismo tipo.
- **Ejemplo clásico (factorial):**

```
def factorial(n):
    if n == 0:
        return 1
    else:
        return n * factorial(n - 1)

resultado = factorial(5) # Devuelve 120
```

- **Precauciones:**

- **Condición de parada:** Es fundamental incluir una condición que detenga la recursión para evitar una recursión infinita y un desbordamiento de pila (stack overflow).
 - **Profundidad de recursión:** Python tiene un límite en la profundidad de recursión que se puede ajustar con `sys.setrecursionlimit()`, pero debe manejarse con cuidado.
-

Resumen sobre Recursividad en Python

1. Concepto

- **Definición:** La recursividad es una técnica en programación donde una función se llama a sí misma para resolver un problema. Es especialmente útil para problemas que pueden ser divididos en subproblemas más pequeños del mismo tipo.
- **Característica clave:** Cada llamada recursiva trabaja en una versión más pequeña del problema original, y se resuelve cuando se llega a una condición base o caso base, que detiene la recursión.

2. Implementación

- **Elementos clave:**
 - **Caso base:** Es la condición que termina la recursión, evitando una llamada infinita. Sin un caso base, la recursión continuaría indefinidamente, lo que podría causar un desbordamiento de pila (stack overflow).
 - **Llamada recursiva:** Es cuando la función se llama a sí misma con una versión modificada del problema original.
- **Ejemplo básico (factorial):**

```
def factorial(n):  
    if n == 0: # Caso base  
        return 1  
    else:  
        return n * factorial(n - 1) # Llamada recursiva  
  
resultado = factorial(5) # Devuelve 120
```

Resumen sobre Tipos Abstractos de Datos (TAD) en Python

1. Concepto

- **Definición:** Un Tipo Abstracto de Datos (TAD) es una descripción matemática de un conjunto de datos y las operaciones que se pueden realizar sobre ellos, sin especificar cómo se implementan

dichas operaciones. Los TAD se enfocan en **qué** operaciones se pueden realizar, no en **cómo** se implementan.

- **Ejemplos comunes:** Pila (stack), Cola (queue), Lista enlazada (linked list), Árbol (tree), y Grafo (graph).

2. Encapsulamiento

- **Definición:** El encapsulamiento es el principio de ocultar los detalles internos de una implementación de un TAD, exponiendo solo las operaciones necesarias para interactuar con los datos. Esto permite cambiar la implementación sin afectar a quienes usan el TAD.
- **Beneficios:**
 - **Modularidad:** Facilita el mantenimiento y la actualización del código.
 - **Protección:** Evita que el usuario acceda o modifique directamente los datos internos, preservando la integridad del TAD.
 - **Interfaz clara:** Los usuarios interactúan con el TAD a través de una interfaz bien definida, sin necesidad de conocer los detalles de la implementación.
- **Ejemplo en Python:**

```
class Pila:
    def __init__(self):
        self.__elementos = [] # Encapsulamiento: lista privada

    def apilar(self, elemento):
        self.__elementos.append(elemento)

    def desapilar(self):
        if not self.esta_vacia():
            return self.__elementos.pop()
        else:
            raise IndexError("La pila está vacía")

    def esta_vacia(self):
        return len(self.__elementos) == 0

# Uso del TAD Pila
pila = Pila()
pila.apilar(1)
pila.apilar(2)
print(pila.desapilar()) # Devuelve 2
```